ML (programming language)

ML (Meta Language) is the metalanguage developed for the Edinburgh LCF theorem prover in the 1970s. It is an early statically typed, functional language with polymorphic type inference in the Hindley–Milner style, and other features like exceptions and mutable variables.[1] ML's design in LCF directly inspired the later ML family (notably Standard ML, Caml, and their derivatives) and influenced subsequent functional language development.[4]

ML	
Paradigm	Multi-paradigm: functional, generic, imperative, interactive
Designed by	Robin Milner, others at the University of Edinburgh
First appeared	1973
Typing discipline	Inferred, static, strong
Dialects	
Dependent ML, Lazy ML, ATS	
Influenced by	
ISWIM, POP-2, PAL, GEDANKEN[1]	
Influenced	
Caml, Clojure, Cyclone, C++, Elm,[2] Erlang, F#, F*, Haskell, Idris, Kotlin, Miranda, Nemerle,[3] OCaml, Opa, Rocq, Rust, Scala, Standard ML	

History

ML started development by Robin Milner upon his arrival at University of Edinburgh in 1973 with the help of research assistants Lockwood Morris and Malcolm Newey, both postdocs from Stanford who were hired by Milner.[5] Michael Gordon, Christopher Wadsworth, and other graduate students joined in research by 1975.[4] Historically, ML was conceived to develop proof tactics in the LCF theorem prover and succeed the previous iteration *Stanford LCF*, trying to solve issues regarding space utilization and proof extensibility. ML acted as both a metalanguage (hence the name) and command (REPL) language for the LCF system. *PPLAMBDA*, a language that was conceptually a combination of the first-order predicate calculus and the simply typed polymorphic lambda calculus, was the underlying language that theorem statements were more directly constructed in.[1]

As ML was being developed, Milner wrote the paper *A theory of type polymorphism in programming* in 1978, which laid out the ideas of what it meant for a program to be well-typed in the context of a polymorphic (generic) type system. He used ML as the case-study application of the theorems developed and noted the theoretical challenges that arose with development that had yet to have been solved.[6] The design of the first version of ML was finalized, and subsequently documented in the 1979 book *Edinburgh LCF* by Milner along with Gordon and Wadsworth.[5][1]

After *Edinburgh LCF* was established with the publication of the same name, interest in the language grew and several implementations, all with slight alterations in design and features, were underway by several parties. Luca Cardelli created *Cardelli ML*, or *VAX ML*, which eventually grew into a standalone dialect fit for general-purpose computing, specified with the paper *ML under Unix*.[7][4] Gérard Huet at Inria began porting the source code from Stanford Lisp to various other dialects of Lisp in "Project Formel". The port to Franz Lisp was further developed by Larry Paulson, whose version eventually was coined *Cambridge LCF*.[8] This version of the LCF was subsequently updated to use an early version of Standard ML, and has been uploaded to GitHub.[9]

With the attention and excitement around ML, LCF, and other related technologies at the time, such as the contemporaneous programming language Hope, happening subsequent to the release of *Edinbugh LCF* and other developments at the time, a meeting was convened with the title "ML, LCF, and Hope" in November 1982. Concerns with the splintering of both design and implementation leading to duplicated work was raised in this meeting. Although Milner seemed open to the spirit of experimentation in the meeting, further discussions and meetings were had between Bernard Sufrin and Milner, where Sufrin urged Milner to unify the design of ML. These correspondences were later referenced in the second draft of Milner's proposal for Standard ML.[4]

Overview

The most notable inspiration of the syntax of ML can be traced to ISWIM, a language that was described as "lambda calculus with syntactic sugar".[4] ML was designed with a strong static type system that allowed the user to define abstract types with parametric polymorphism and was checked at compile-time.[1] It also had automatic type inference, which afforded ML the ease-of-use of dynamic languages of the time such as Lisp or POP-2 by foregoing the need for explicit type annotations.[4]

Examples

The following examples are very closely derived from *Edinburgh LCF*, showing a rough overview of the syntax and features of ML. The `#` character at the start of a line denotes user input, and lines without it are system responses showing the value and its inferred type. Note that this section is not meant to act as a comprehensive set of language features that ML contains, rather a subset to give a sense for the language. Refer to *Edinburgh LCF* for a more rigorous definition.[1]

Expressions are evaluated by typing them followed by `;;` and a return character. The identifier `it` holds the result of the last-evaluated expression. Bindings are introduced with `let`, and multiple bindings can be made simultaneously by joining them with the `and` keyword, or by constructing pairs (which has a product type, explored in a later example) on the right hand side, which is pattern-matched to the left:

```
#2+3;;
5 : int

#let x = it;;
x = 5 : int

#let y = 2*5 and z = 7;;
y = 10 : int
z = 7 : int

#let x,y,z = y,x,2;;
x = 10 : int
y = 5 : int
z = 2 : int
```

Functions are defined with `let`. Function application is higher precedent than mathematical operators, so `f 3 + 4` means `(f 3) + 4`. Functions defined with multiple parameters are curried, so passing one parameter into the function will return a function that will accept the second, and so on. Recursive

functions require `letrec` so the function name is in scope within its body. The syntax for an anonymous function is similar to lambda calculus, with `\` for lambda and `.` separating arguments from the expression:

```
#let add x y = x+y;;
add = - : (int -> (int -> int))

#add 3;;
- : (int -> int)

#it 4;;
7 : int

#letrec fact n = if n = 0 then 1 else n * fact(n-1);;
fact = - : (int -> int)

#fact 4;;
24 : int

#(\x.x+1) 3;;
4 : int
```

Lists use semicolons between elements. `hd` and `tl` are built-in functions that return the head and tail; `.` is cons (prepend); `@` is append. Functions like `hd` are polymorphic—ML uses generic type variables (`*`, `**`, etc.) to express this:

```
#let m = [1;2;3;4];;
m = [1; 2; 3; 4] : (int list)

#hd m, tl m;;
1, [2; 3; 4] : (int # (int list))

#0.m @ [5;6];;
[0; 1; 2; 3; 4; 5; 6] : (int list)

#hd;;
- : ((* list) -> *)

#map (\x.x*x) [1;2;3;4];;
[1; 4; 9; 16] : (int list)
```

Mutable variables are declared with `letref` and updated with `:=`. The `loop` keyword belongs to the if-then-loop construct, which iterates every time the `if` conditional fails:

```
#let fact n =
#    letref count = n and result = 1
#    in     if count = 0
#           then result
#           loop count,result := count-1, count*result;;
fact = - : (int -> int)

#fact 4;;
24 : int
```

Tokens are ML's string type, delimited by `` ` ``; double backticks create token lists. A common use for tokens was identifying failures with `failwith`, a keyword used to raise exceptions with an explicit token, and `?` is used to trap it:

```
#`this is a token`;;
`this is a token` : tok

#``this is a token list``;;
```

```
[`this`; `is`; `a`; `token`; `list`] : (tok list)

#let half n =
#    if n = 0 then failwith `zero`
#             else let m = n/2
#                  in if n = 2*m then m else failwith `odd`;;
half = - : (int -> int)

#half 4;;
2 : int

#half 3;;
EVALUATION FAILED odd

#half 3 ? 0;;
0 : int
```

Abstract types are declared with `abstype`, which creates a new type and defines the functions that work with it, while keeping the internal structure (the concrete type it's built from) hidden. Abstract recursive types are declared with `absrectype`, which allows the type to be used in its own definition. There is a similar keyword `lettype`, which is used for more basic type aliasing. Here is a recursive abstract type that defines a binary tree with some basic operations:

```
#absrectype (*, **) tree = * + ** # (*, **) tree # (*, **) tree
#    with tiptree x = abstree(inl x)
#    and comptree (y, t1, t2) = abstree(inr(y, t1, t2))
#    and istip t = isl(reptree t)
#    and tipof t = outl(reptree t) ? failwith `tipof`
#    and labelof t = fst(outr(reptree t)) ? failwith `labelof`
#    and sonsof t = snd(outr(reptree t)) ? failwith `sonsof`;;
tiptree = - : (* -> (*, **) tree)
comptree = - : ((** # (*, **) tree # (*, **) tree) -> (*, **) tree)
istip = - : ((*, **) tree -> bool)
tipof = - : ((*, **) tree -> *)
labelof = - : ((*, **) tree -> **)
sonsof = - : ((*, **) tree -> ((*, **) tree # (*, **) tree))
```

Inside the operation definitions (the `with` block), the type's name can be prepended with `abs` to box values in the type and `rep` to unbox values. The characters `+` and `#` in type definitions signify sum types (tagged unions) and product types (tuples), respectively, with `#` having a higher precedence.

ML on LCF provided several helper functions used in the example above. Operating on sum types `+`, the functions `inl` and `inr` inject values into the left or right side of a sum type. `outl` extracts from left injections (fails if given a right injection), and `outr` extracts from right injections (fails if given a left injection). For product types `#`, the extraction functions are `fst` and `snd`, which extract the first and second components of a pair. Product types are constructed using the comma infix operator. In the tree example above, `comptree` takes a single parameter that is a pair pattern `(y, t1, t2)`, which destructures the pair into its three components.

See also

- HOL (proof assistant)
- Hope (programming language)
- Isabelle (proof assistant)
- Nuprl

References

1. Gordon, M.; Milner, R.; Wadsworth, C. P. (1979). *Edinburgh LCF: A Mechanized Logic of Computation*. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg. doi:10.1007/3-540-09724-4 (https://doi.org/10.1007%2F3-540-09724-4). ISBN 978-3-540-09724-2.
2. Tate, Bruce; Dees, Ian; Daoud, Frederic; Moffitt, Jack (November 18, 2014). *Seven more languages in seven weeks: languages that are shaping the future*. Pragmatic programmers (P1.0 ed.). Dallas: The Pragmatic Bookshelf. pp. 97, 101. ISBN 978-1-941222-15-7. "I tend to say 'Elm is an ML-family language' to get at the shared heritage of all these languages [Haskell, OCaml, SML, F#]."
3. *Programming language for "special forces" of developers* (http://nemerle.org/About), Russian Software Development Network: Nemerle Project Team, retrieved January 24, 2021
4. MacQueen, David; Harper, Robert; Reppy, John (2020-06-14). "The history of Standard ML" (https://dl.acm.org/doi/10.1145/3386336). *Proceedings of the ACM on Programming Languages*. **4** (HOPL): 1–100. doi:10.1145/3386336 (https://doi.org/10.1145%2F3386336). ISSN 2475-1421 (https://search.worldcat.org/issn/2475-1421).
5. Gordon, Michael J. C. (1996). "From LCF to HOL: a short history" (http://www.cl.cam.ac.uk/~mjcg/papers/HolHistory.html). Retrieved 2007-10-11.
6. Milner, Robin (December 1978). "A theory of type polymorphism in programming" (https://linkinghub.elsevier.com/retrieve/pii/0022000078900144). *Journal of Computer and System Sciences*. **17** (3): 348–375. doi:10.1016/0022-0000(78)90014-4 (https://doi.org/10.1016%2F0022-0000%2878%2990014-4).
7. Cardelli, Luca (1983). "ML under Unix" (https://web.archive.org/web/20251112171739/http://lucacardelli.name/Papers/MLUnix.pdf) (PDF). Archived from the original (http://lucacardelli.name/Papers/MLUnix.pdf) (PDF) on November 12, 2025. Retrieved January 25, 2025.
8. Gordon, Michael J. C. (1988), Birtwistle, Graham; Subrahmanyam, P. A. (eds.), "HOL: A Proof Generating System for Higher-Order Logic" (http://link.springer.com/10.1007/978-1-4613-2007-4_3), *VLSI Specification, Verification and Synthesis*, vol. 35, Boston, MA: Springer US, pp. 73–128, doi:10.1007/978-1-4613-2007-4_3 (https://doi.org/10.1007%2F978-1-4613-2007-4_3), ISBN 978-1-4612-9197-8, retrieved 2026-01-03
9. Kohlhase, Michael (2024-05-11), *kohlhase/CambridgeLCF* (https://github.com/kohlhase/CambridgeLCF), retrieved 2026-01-03

Further reading

- Christoph Kreitz, Vincent Rahli, *Introduction to Classic ML* (https://www.cs.uni-potsdam.de/ti/kreitz/PDF/11cucs-ClassicML.pdf) (Archived July 8, 2024 (https://web.archive.org/web/20240708040921/https://www.cs.uni-potsdam.de/ti/kreitz/PDF/11cucs-ClassicML.pdf)), Cornell University, October 2011. Lecture notes on a dialect of ML close in spirit to *LCF/ML*.

- Luca Cardelli, *Papers* (http://lucacardelli.name/indexpapers.html) (Archived January 25, 2026 (https://web.archive.org/web/20260125231118/http://lucacardelli.name/indexpapers.html)), Oxford University. Lists papers concerning the development of Cardelli ML/ML under VMS/ML under Unix.

Retrieved from "https://en.wikipedia.org/w/index.php?title=ML_(programming_language)&oldid=1338344190"